MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Gabriela Pricop, FAF-241

Report

Laboratory Work No. 1 & 2

Operating Systems

Checked by:
Reițman Patricia, *university assistant*

Chișinău – 2026

Contents

1 Part 1: Files and Directories **3**

2 Part 2: Process Management **6**

3 Part 3: Memory **9**

4 Part 4: Device Storage I/O management **10**

5 Laboratory Work 2 **12**

6 Part 1: Built it and boot it **12**

7 Part 2: Use xv6 as the Unix it is **12**

8 Part 3: Read the source **14**

9 Part 4: Write your first xv6 program **16**

Part 1: Files and Directories

Questions:

- Who owns the files you just created, and which group?
- What do the ten characters at the start of 'ls -l' mean for note.txt after chmod 600?
- Name two directories directly under / and say, in one line each, what they hold.

Answers:

- The owner is precision_5540 and the group has the same name precision_5540
- The 10 characters are about permissions, the first - means it is a regular file, rw- means that the user can read and write, — group has no permissions, — others have no permissions
- Folders: /etc contains system configuration files, and the /bin contains the essential user command binaries

```
precision_5540@DELL:/mnt/c/Users/Precision 5540$ mkdir -p ~/os-lab1 && cd ~/
os-lab1
precision_5540@DELL:~/os-lab1$ whoami
precision_5540
precision_5540@DELL:~/os-lab1$ uname -a
Linux DELL 6.18.40.1-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Fri Jul
31 22:12:15 UTC 2026 x86_64 GNU/Linux
precision_5540@DELL:~/os-lab1$ uptime
 18:39:32 up 18 min,  1 user,  load average: 0.07, 0.04, 0.00
precision_5540@DELL:~/os-lab1$
```

Figure 1.1: Commands output

mkdir -p ~/os-lab1 && cd ~/os-lab1 – first creates the directory `os-lab1` inside the user's home folder if it does not already exist. The option `-p` ensures that no error is raised if the directory is already present. After creating the directory, the command changes the current working directory to `os-lab1`, preparing the workspace for laboratory tasks.

whoami – prints the username of the current user. It shows how the operating system identifies the person running commands, which is useful for confirming permissions and ownership of files or processes.

uname -a – displays detailed information about the system kernel and operating environment. The output includes the kernel name, version, release date, machine architecture, and sometimes the hostname. This helps to understand which kernel is managing the machine and provides context about the operating system version.

uptime – shows how long the system has been running since the last boot. It also displays the current time, the number of users logged in, and the system load averages. This information is useful for monitoring system stability and performance over time.

```
precision_5540@DELL:~/os-lab1$ pwd
/home/precision_5540/os-lab1
precision_5540@DELL:~/os-lab1$ ls -la /
total 3420
drwxr-xr-x  19 root root    4096 Oct  3 18:37 .
drwxr-xr-x  19 root root    4096 Oct  3 18:37 ..
lrwxrwxrwx   1 root root       7 Apr 20 11:46 bin -> usr/bin
drwxr-xr-x   2 root root    4096 Apr 20 11:46 boot
drwxr-xr-x  14 root root    3820 Oct  3 18:37 dev
drwxr-xr-x  89 root root    4096 Oct  5 07:55 etc
drwxr-xr-x   3 root root    4096 Sep 28 17:41 home
-rwxr-xr-x   1 root root 3430904 Sep 26 02:31 init
lrwxrwxrwx   1 root root       7 Apr 20 11:46 lib -> usr/lib
lrwxrwxrwx   1 root root       9 Apr 20 11:46 lib64 -> usr/lib64
drwx------   2 root root   16384 Sep 28 17:41 lost+found
drwxr-xr-x   2 root root    4096 Aug 27 17:10 media
```

Figure 1.2: pwd and ls -la commands

pwd – prints the current working directory. It shows the exact location in the filesystem where the user is positioned. This is useful to confirm the path before running other commands that depend on the current directory.

ls -la / – lists all files and directories in the root of the filesystem. The options `-l` and `-a` display detailed information including permissions, owner, group, size, and hidden files. This helps to understand the structure of the root directory.

```
precision_5540@DELL:~/os-lab1$ ls -la ~
total 36
drwxr-x--- 5 precision_5540 precision_5540 4096 Sep 30 16:08 .
drwxr-xr-x 3 root           root           4096 Sep 28 17:41 ..
-rw------- 1 precision_5540 precision_5540  306 Oct  3 18:36 .bash_history
-rw-r--r-- 1 precision_5540 precision_5540  220 Sep 28 17:41 .bash_logout
-rw-r--r-- 1 precision_5540 precision_5540 3771 Sep 28 17:41 .bashrc
drwxr-x--- 4 precision_5540 precision_5540 4096 Sep 28 17:42 .cache
```

Figure 1.3: ls -la command

ls -la ~ – lists all files and directories in the home directory of the current user. The tilde symbol ~ is shorthand for the home path. This shows personal files and configuration directories with detailed information.



Figure 1.4: cd /etc && ls | head command

cd /etc && ls | head – first changes the directory to `/etc`, which contains system configuration files. Then it lists the contents of that directory and pipes the output to `head`, showing only the first ten entries. This is useful for quickly checking the beginning of a long directory listing.

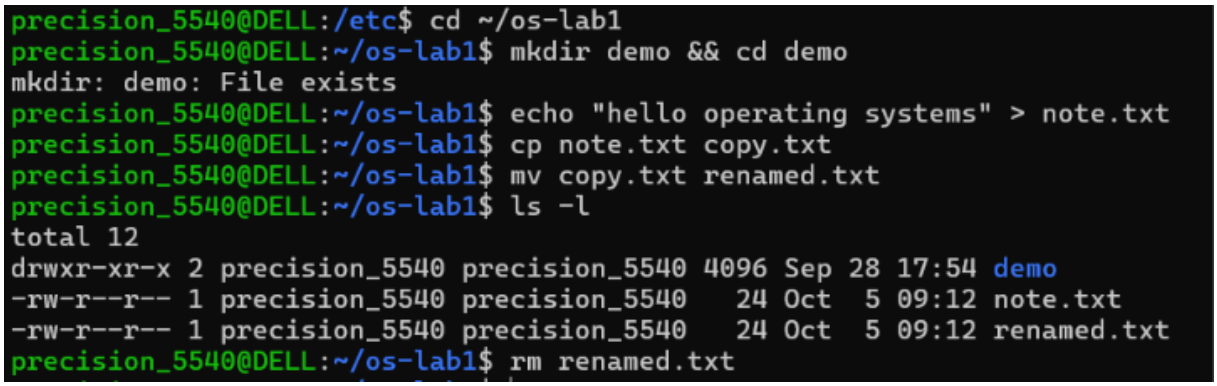


Figure 1.5: File management commands

cd ~/os-lab1 – changes the current working directory to the folder named `os-lab1` located in the user's home directory. This is often used to move into the workspace prepared for laboratory tasks.

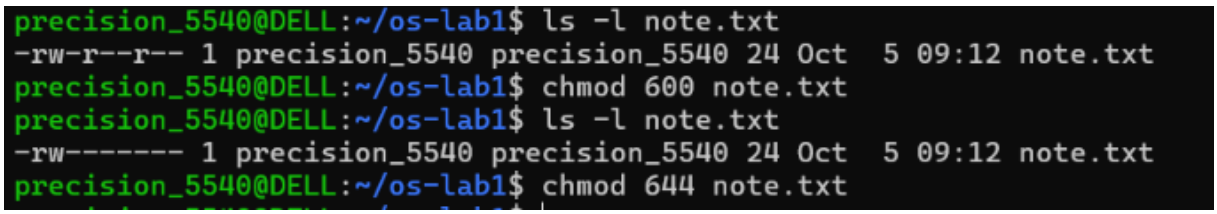


Figure 1.6: File permissions commands

ls -l – lists the files in the current directory in long format. It shows detailed information such as permissions, owner, group, file size, and modification date. This helps to understand file attributes beyond just their names.

ls -l note.txt – displays detailed information about the file `note.txt`. The output includes the permissions (rwx), owner, group, size, and modification time. This is useful for checking who can read or write the file.

chmod 600 note.txt – changes the permissions of `note.txt` so that only the owner has read and write access. Other users have no permissions. This is often used to protect sensitive files.

chmod 644 note.txt – sets the file permissions so that the owner can read and write, while the group and others can only read. This is a common setting for text files that should be readable by everyone but only editable by the owner.

Part 2: Process Management

Questions:

- What is the PID of process number 1, and what is it (look it up)?
- Roughly how many processes were running on your idle VM?
- From /proc/<pid>/status, what does the "State:" line say for your sleeping process?

Answers:

- The PID of process number one is 1, which is init
- 26 background processes were running
- State for the sleeping process is S (sleeping)

```
precision_5540@DELL:~/os-lab1/demo$ ps aux | head
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.0  24728 15684 ?        Ss   17:41   0:00 /sbin/ini
t
root           2  0.0  0.0   3180  2196 hvc0     Sl+  17:41   0:00 /init
root           7  0.0  0.0   3212  2136 hvc0     Sl+  17:41   0:00 plan9 --c
ontrol-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-truncate
root          57  0.0  0.1  50396 16888 ?        S<s  17:41   0:00 /usr/lib/
systemd/systemd-journald
systemd+      94  0.0  0.0  22416 14444 ?        Ss   17:41   0:00 /usr/lib/
systemd/systemd-resolved
root         101  0.0  0.0  35484 12460 ?        Ss   17:41   0:00 /usr/lib/
systemd/systemd-udevd
root         126  0.0  0.0   2888  1948 ?        Ss   17:41   0:00 /bin/sh /
usr/lib/systemd/scripts/chronyd-starter.sh -n -F 1
root         127  0.0  0.0   4472  3128 ?        Ss   17:41   0:00 /usr/sbin
/cron -f -P
message+     128  0.0  0.0   8832  5388 ?        Ss   17:41   0:00 @dbus-dae
mon --system --address=systemd: --nofork --nopidfile --systemd-activation --
syslog-only
precision_5540@DELL:~/os-lab1/demo$ ps aux | wc -l
26
```

Figure 2.1: Commands output

ps aux | head – lists all running processes with detailed information such as user, PID, CPU usage, memory usage, and command. The output is then piped to `head`, which shows only the first ten lines. This is useful for quickly checking the beginning of the process list.

ps aux | wc -l – counts the number of processes currently running on the system. The `ps aux` command lists all processes, and `wc -l` counts the lines of output, giving the total number of processes.

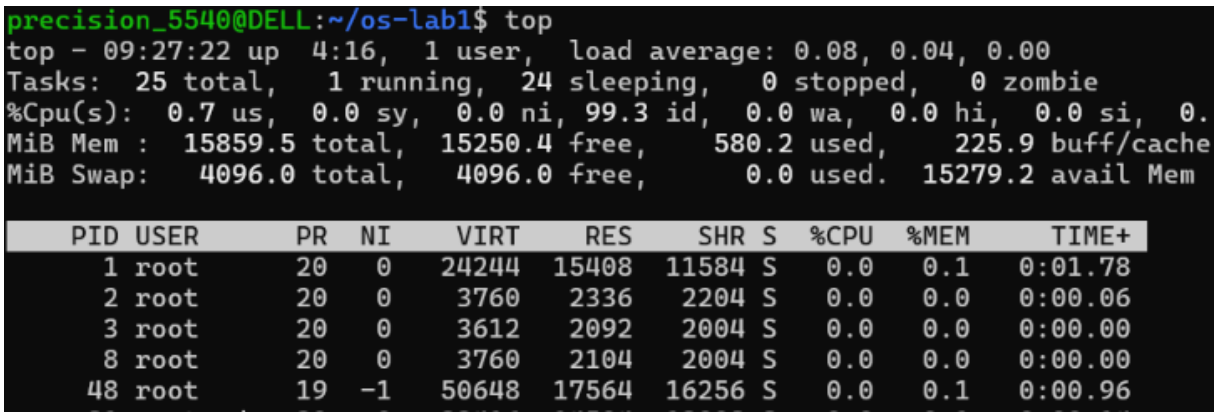


Figure 2.2: Top command

top – provides a live view of system processes. It shows CPU and memory usage, process IDs, users, and commands in real time. This tool is useful for monitoring system performance and identifying processes that consume the most resources. Pressing `q` exits the view.

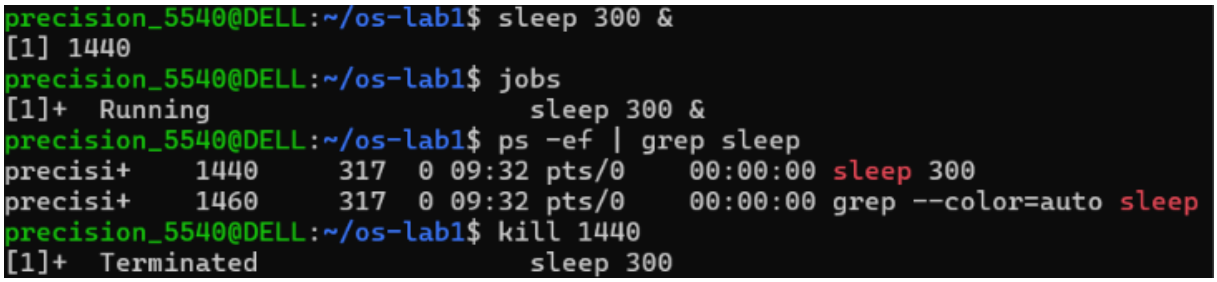


Figure 2.3: Working with background processes

sleep 300 & – runs the `sleep` program in the background, pausing for 300 seconds. The ampersand `&` places the process in the background and prints its PID, allowing the user to continue using the shell while the process runs.

jobs – lists the background jobs started in the current shell session. It shows their job numbers, status, and the command that launched them. This is useful for tracking background tasks.

ps -ef | grep sleep – lists all processes in full format and filters the output to show only those containing the word `sleep`. This is a common way to locate specific processes by name.

kill <PID> – sends a termination signal to the process with the given PID. This asks the operating system to stop the process. It is often used to end background jobs or unresponsive programs.

```
precision_5540@DELL:~/os-lab1$ sleep 300 &
[1] 1507
precision_5540@DELL:~/os-lab1$ PID=$!
precision_5540@DELL:~/os-lab1$ ls /proc/$PID/
arch_status       gid_map             oom_adj        stack
attr              ksm_merging_pages   oom_score      stat
auxv              ksm_stat            oom_score_adj  statm
cgroup            limits              pagemap        status
clear_refs        loginuid            patch_state    syscall
cmdline           map_files           personality    task
comm              maps                projid_map     timens_offsets
coredump_filter   mem                 root           timers
cpuset            mountinfo           sched          timerslack_ns
cwd               mounts              schedstat      uid_map
environ           mountstats          sessionid      wchan
exe               net                 setgroups
fd                ns                  smaps
fdinfo            numa_maps           smaps_rollup
```

Figure 2.4: Storing recent background processes

ls /proc/$PID/ – lists the contents of the `/proc` directory for the process with the given PID. This directory contains information about the process such as its memory, status, and file descriptors.

```
precision_5540@DELL:~/os-lab1$ cat /proc/$PID/status | head -20
Name:   sleep
Umask:  0022
State:  S (sleeping)
Tgid:   1507
Ngid:   0
Pid:    1507
PPid:   317
TracerPid:      0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
FDSize: 256
Groups: 4 24 27 30 46 100 1000
NStgid: 1507
NSpid:  1507
NSpgid: 1507
```

Figure 2.5: Command output

cat /proc/$PID/status | head -20 – displays the first twenty lines of the status file for the process with the given PID. This file includes details such as the process state, memory usage, and number of threads.

Part 3: Memory

Questions:

- How much total RAM does the VM have, and how much is free?
- What is swap, and how much is configured?
- How much resident memory (VmRSS) does a bare ‘sleep‘ process use, and does that surprise you?

Answers:

- Total RAM is MemTotal, which is 16240096 kB 15-16 GB, free RAM is MemFree, 15613040 kB, 15 GB
- Swap is disk space reverved to be used as additional RAM in case of need. Configured Swap 4.0Gi.
- The sleep process uses 7932kB of VmRSS, which is 7.9MB of RAM. Evem if it is a sleep process it still uses some memory because of the overhead, code, libraries, environments

```
precision_5540@DELL:~/os-lab1$ free -h
               total        used        free      shared  buff/cache   avail
able
Mem:            15Gi       583Mi        14Gi       4.1Mi       226Mi
14Gi
Swap:          4.0Gi          0B       4.0Gi
precision_5540@DELL:~/os-lab1$ cat /proc/meminfo | head -6
MemTotal:       16240096 kB
MemFree:        15613040 kB
MemAvailable:   15643044 kB
Buffers:           14964 kB
Cached:           195276 kB
SwapCached:            0 kB
precision_5540@DELL:~/os-lab1$ sleep 300 & PID=$!
[1] 1578
precision_5540@DELL:~/os-lab1$ grep VmRSS /proc/$PID/status
VmRSS:      7932 kB
```

Figure 3.1: Memory usage

free -h – displays information about the system’s memory usage. It shows the total, used, and free RAM, as well as swap space. The option `-h` formats the output in human-readable units such as MB or GB, making it easier to interpret.

cat /proc/meminfo | head -6 – prints the first six lines of the `/proc/meminfo` file, which contains detailed information about system memory. This includes total memory, free memory, available memory, and other statistics. Limiting the output with `head -6` makes it easier to focus on the most important values.

sleep 300 & PID=$! – runs the `sleep` program in the background for 300 seconds and stores its process ID in the variable `PID`. This allows the user to track and manage the process later using the saved PID.

grep VmRSS /proc/$PID/status – searches the status file of the process with the given PID for the line containing `VmRSS`. This value represents the actual physical RAM used by the process. It is useful for monitoring memory consumption of a specific program.

Part 4: Device Storage I/O management

Questions:

- Which device is your root filesystem "/" mounted on?
- Give one entry from /dev and say what real thing it stands for.
- In one sentence: what does "everything is a file" mean, based on what you saw?

Answers:

- The root filesystem is mounted on /dev/sde.
- One entry from dev is console which is the crw——- 1 root root 5, it is the system console device file, which is the physical/virtual terminal interface.
- Everything is a file means the OS represents devices, processes, and resources as files in the filesystem, so you can interact with them using commands you use with text files.

```
precision_5540@DELL:~/os-lab1$ df -h
Filesystem      Size  Used Avail Use% Mounted on
none            7.8G     0  7.8G   0% /usr/lib/modules/6.18.40.1-microsoft-s
tandard-WSL2
none            7.8G  4.0K  7.8G   1% /mnt/wsl
drivers         250G  225G   25G  91% /usr/lib/wsl/drivers
/dev/sde       1007G  1.4G  955G   1% /
none            7.8G   60K  7.8G   1% /mnt/wslg
none            7.8G     0  7.8G   0% /usr/lib/wsl/lib
rootfs          7.8G  3.3M  7.8G   1% /init
none            7.8G  520K  7.8G   1% /run
none            7.8G     0  7.8G   0% /run/lock
none            7.8G     0  7.8G   0% /run/shm
none            7.8G   80K  7.8G   1% /mnt/wslg/versions.txt
none            7.8G   80K  7.8G   1% /mnt/wslg/doc
C:\             250G  225G   25G  91% /mnt/c
D:\             226G  112G  115G  50% /mnt/d
tmpfs           7.8G     0  7.8G   0% /tmp
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.serv
```

Figure 4.1: Mounted filesystems and free space

df -h – displays information about mounted filesystems, including their total size, used space, available space, and mount points. The option `-h` makes the output human-readable by showing sizes in MB or GB. This is useful for checking disk usage across different partitions.

```
precision_5540@DELL:~/os-lab1$ lsblk
NAME MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda    8:0    0 722.9M  1 disk
sdb    8:16   0  96.5M  1 disk
sdc    8:32   0     4G  0 disk [SWAP]
sdd    8:48   0 128.6M  0 disk
sde    8:64   0     1T  0 disk /mnt/wslg/distro
```

Figure 4.2: The block devices (disks) themselves

lsblk – lists block devices such as hard drives, SSDs, and partitions. It shows their names, sizes, types, and mount points. This provides a clear overview of the physical storage devices connected to the system and how they are organized.

```
precision_5540@DELL:~/os-lab1$ du -sh ~/os-lab1
16K     /home/precision_5540/os-lab1
precision_5540@DELL:~/os-lab1$ ls -l /dev | head
total 0
crw-r--r-- 1 root root     10, 235 Oct  3 18:37 autofs
drwxr-xr-x 2 root root         620 Oct  3 18:37 block
drwxr-xr-x 2 root root         140 Oct  3 18:22 bsg
crw------- 1 root root     10, 234 Oct  3 18:23 btrfs-control
drwxr-xr-x 2 root root        2860 Oct  3 18:37 char
crw------- 1 root root      5,   1 Oct  5 07:55 console
lrwxrwxrwx 1 root root          11 Oct  3 18:22 core -> /proc/kcore
crw------- 1 root root     10, 260 Oct  3 18:37 cpu_dma_latency
crw------- 1 root root     10, 203 Oct  3 18:23 cuse
```

Figure 4.3: Size of the work folder

du -sh ~/os-lab1 – calculates the total size of the directory `os-lab1` in the user's home folder. The options `-s` and `-h` summarize the result and display it in human-readable format. This is useful for measuring how much space a project or folder occupies.

ls -l /dev | head – lists the first ten entries of the `/dev` directory in long format. The `/dev` directory contains device files that represent hardware devices such as disks, terminals, and input/output interfaces. This shows how Linux presents devices as files.

```
precision_5540@DELL:~/os-lab1$ mount | head
none on /usr/lib/modules/6.18.40.1-microsoft-standard-WSL2 type overlay (rw,
nosuid,nodev,noatime,lowerdir=/modules/6.18.40.1-microsoft-standard-WSL2/mod
ules,upperdir=/lib/modules/6.18.40.1-microsoft-standard-WSL2/rw/upper,workdi
r=/lib/modules/6.18.40.1-microsoft-standard-WSL2/rw/work,uuid=on)
none on /mnt/wsl type tmpfs (rw,relatime)
drivers on /usr/lib/wsl/drivers type 9p (ro,nosuid,nodev,noatime,aname=drive
rs;fmask=222;dmask=222,cache=0x5,access=client,msize=65536,trans=fd,rfd=8,wf
d=8)
/dev/sde on / type ext4 (rw,relatime,discard,errors=remount-ro,data=ordered)
none on /mnt/wslg type tmpfs (rw,relatime)
/dev/sde on /mnt/wslg/distro type ext4 (ro,relatime,discard,errors=remount-r
```

Figure 4.4: Quick peek at active mounts

mount | head – displays the first ten lines of the list of mounted filesystems. Each line shows the device, mount point, filesystem type, and options. This helps to understand which filesystems are currently active and how they are connected to the system.

Laboratory Work 2

Part 1: Built it and boot it

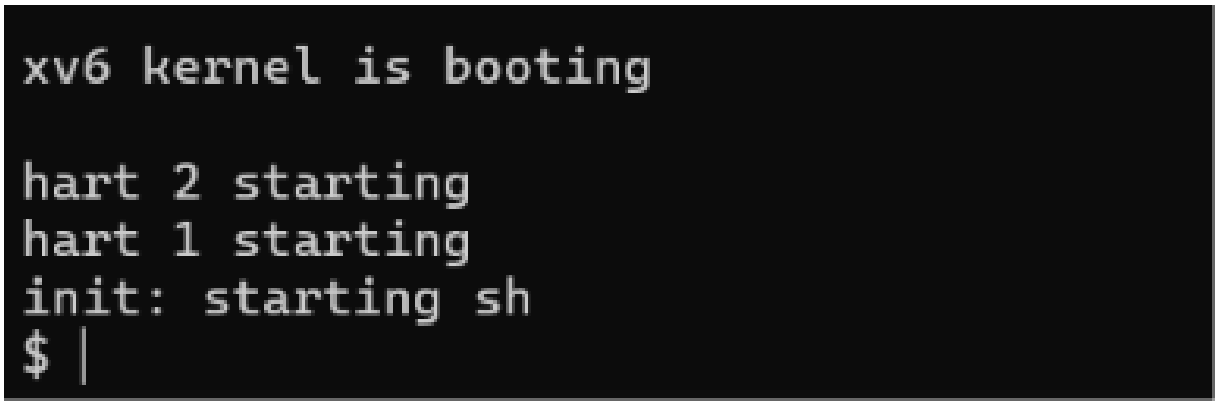


Figure 6.1: Kernel is booting

QEMU Installation and xv6 Kernel Booting The installation of QEMU was required to emulate the xv6 operating system. After setting up QEMU, the xv6 source code was compiled using the `make qemu` command, which built the kernel and launched the emulator. During booting, the xv6 kernel initialized its subsystems such as memory management, process scheduling, and device drivers, and then presented the user with a shell prompt inside the virtual environment. This process demonstrated how an operating system kernel is loaded and executed in a controlled environment, making it possible to run and test user programs without affecting the host system.

Part 2: Use xv6 as the Unix it is

Questions:

- List three programs xv6 ships with.
- You just used a pipe (|) inside xv6. Which two OS features must exist for a pipe to work? (Hint: think processes and communication.)
- In one sentence: how does the xv6 shell compare to the Linux shell from Lab 1?

Answers:

- Three programs xv6 ships with are ls, echo, cat
- Two OS features must exist for a pipe to work and those are process management (capable to run multiple processes concurrently) and inter-process communication (to pass data between processes)

- xv6 shell is more simple implementation of a unix shell, but the shell from lab 1 is more complex, has more features.

```
precision_5540@DELL:~/xv6-riscv$ ls
LICENSE  Makefile  README  fs.img  kernel  mkfs  test-xv6.py  user
precision_5540@DELL:~/xv6-riscv$ cat README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6).  xv6 loosely follows the structure and style of v6,
but is implemented for a modern RISC-V multiprocessor using ANSI C.

ACKNOWLEDGMENTS

xv6 is inspired by John Lions's Commentary on UNIX 6th Edition (Peer
to Peer Communications; ISBN: 1-57398-013-7; 1st edition (June 14,
2000)).  See also https://pdos.csail.mit.edu/6.1810/, which provides
pointers to on-line resources for v6.

The following people have made contributions: Russ Cox (context switching,
locking), Cliff Frey (MP), Xiao Yu (MP), Nickolai Zeldovich, and Austin
Clements.
```

Figure 7.1: Contents of readme file

```
precision_5540@DELL:~/xv6-riscv$ echo hello xv6ls
hello xv6ls
precision_5540@DELL:~/xv6-riscv$ ls | grep c
precision_5540@DELL:~/xv6-riscv$ wc README
  48  336 2441 README
```

Figure 7.2: Word count of README and more

ls | grep c The command `ls | grep c` lists the files in the current directory and then filters the output using `grep` to show only those entries that contain the letter `c`. This demonstrates how pipes can be used to connect commands together, allowing one command's output to be processed by another.

wc README – counts the number of lines, words, and characters in the file named `README`. The output provides three values: lines, words, and bytes. This is useful for quickly measuring the size and content of a text file.

Part 3: Read the source

Questions:

- Which system calls does user/cat.c use, and what does each one ask the kernel to do?
- In which kernel file, and on which line, is sys$_{r}$*eadimplemented*?
- In one sentence: what is the difference between the code in kernel/ and the code in user/?

Answers:

- The system calls user/cat.c has are: read (asks the kernel to fetch data from a file descriptor into a buffer in user space), write (asks the kernel to send data from the buffer to a file descriptor), fprintf (writes error messages to file descriptor), exit(terminates the process and returns a status code to the kernel)
- The sys_read is implemented inside kernel/sysfile.c, with the line number printed by grep -n.
- The code in kernel/ runs in privileged mode and implements the operating system itself, while the code in user/ consists of ordinary programs that rely on system calls to request services from the kernel.

```
precision_5540@DELL:~/xv6-riscv$ sed -n "1,40p" user/ls.c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

char *
fmtname(char *path)
{
  static char buf[DIRSIZ + 1];
  char *p;

  // Find first character after last slash.
  for (p = path + strlen(path); p >= path && *p != '/'; p--)
    ;
  p++;
```

Figure 8.1: User Program 1

```
precision_5540@DELL:~/xv6-riscv$ sed -n "1,30p" user/cat.c
#include "kernel/types.h"
#include "kernel/fcntl.h"
#include "user/user.h"

char buf[512];

void
cat(int fd)
{
  int n;

  while ((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
      fprintf(2, "cat: write error\n");
      exit(1);
    }
  }
  if (n < 0) {
    fprintf(2, "cat: read error\n");
    exit(1);
```

Figure 8.2: User Program 2

sed -n "1,40p" user/ls.c – prints lines 1 through 40 from the file `user/ls.c`. The option `-n` suppresses automatic printing, and the `p` command explicitly tells `sed` to print the selected range. This is useful for viewing a specific portion of a source file.

sed -n "1,30p" user/cat.c – prints lines 1 through 30 from the file `user/cat.c`. This allows inspection of the beginning of the source code without opening the entire file, which is helpful for quickly checking function definitions or headers.

cat user/user.h – This file contains the list of system call prototypes available to user programs in xv6. Viewing it helps to understand which kernel functions can be invoked from user space.

```
precision_5540@DELL:~/xv6-riscv$ grep -n "sys_read\|sys_write" kernel/sysfil
e.c | head
69:sys_read(void)
83:sys_write(void)
```

Figure 8.3: System read and System write

grep -n "sys_read\|sys_write" kernel/sysfile.c | head – searches the file `kernel/sysfile.c` for lines containing either `sys_read` or `sys_write`. The option `-n` shows the line numbers where matches occur, and `head` limits the output to the first ten matches. This is useful for locating the implementation of system calls related to file input and output.

Part 4: Write your first xv6 program

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // system call into the kernel
  exit(0);
}
```

Figure 9.1: Wrote program in sleep.c

```
UPROGS=\
        $U/_cat\
        $U/_echo\
        $U/_sleep\
        $U/_forktest\
        $U/_grep\
        $U/_init\
```

Figure 9.2: Added the sleep command

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ sleep 10
$ sleep
usage: sleep <ticks>
$
```

Figure 9.3: Program output

Conclusion

The laboratory work started with an introduction to basic Linux operations on files and directories, process management, memory, and device storage I/O. The first part showed how to list file information, create new files, copy them, write content, and delete them. Commands such as `ls`, `cat`, `echo`, and `wc` were used to explore how files are organized and how information can be displayed or measured. The second part focused on process management, where commands were used to create and kill processes, showing how the operating system controls execution. The memory section explained how programs use memory and how the system allocates and manages it. The device storage I/O section demonstrated how files are read and written, highlighting the role of the operating system in managing input and output. These exercises gave a practical view of how Linux commands interact with the underlying system and made it clear how important these basic operations are for everyday computing. The second laboratory continued with xv6, an educational operating system, and introduced QEMU as the emulator to run it. The work showed how xv6 is split into two main folders: `kernel/`, which contains the privileged code such as the scheduler, memory management, file system, and system call handlers, and `user/`, which holds ordinary programs like `ls`, `cat`, `echo`, and the shell. Commands were used to read user programs and to trace system calls back into the kernel. A new user program called `sleep` was created, added to the Makefile, and tested inside xv6. This program paused execution for a number of clock ticks by calling the sleep system call, showing how user code requests services from the kernel. There were some struggles with writing and compiling the program, especially because the latest version of xv6 actually uses the `pause` function instead of the `sleep` command, but the process helped to understand the boundary between user space and kernel space. Overall, both labs were useful for learning more about Linux commands, xv6, QEMU, and the way operating systems are organized, while also giving practical experience with building and testing code in a real environment.

References

1. *Website to type the command and get help text* `https://explainshell.com/`
2. *Introduction of Process Management* `https://shorturl.at/7vR8G`
3. *Memory Management in Operating System* `https://shorturl.at/uSdaP`
4. *Storage Structure in Operating Systems* `https://shorturl.at/ZeFBW`
5. *xv6 purpose information* `https://en.wikipedia.org/wiki/Xv6`
6. *Adding user programs to xv6* `https://shorturl.at/ar8Xw`
7. *Qemu documentation* `https://github.com/qemu/QEMU`
8. *System Calls - Linux Documentation* `https://shorturl.at/GAVYz`